



CONCORRÊNCIA E MULTITHREADING

AULA 07 - PARALLEL STREAMS



AGENDA

- ❑ Parallel Streams: Paralelismo Funcional e Simplificado
- ❑ O poder e o perigo de `.parallel()`.
- ❑ Quando e como usar streams paralelos de forma segura e eficaz.



A LIGAÇÃO COM FORK/JOIN

- ❑ Mecanismo Interno: Parallel Streams utilizam o `ForkJoinPool.commonPool()` por baixo dos panos.
- ❑ Splitter: A fonte de dados (ex: `ArrayList`) fornece um `Splitter`, que é responsável por dividir a fonte em pedaços para processamento paralelo.
- ❑ Processamento: O `commonPool` executa as operações do stream em cada pedaço, usando o algoritmo de work-stealing.



QUANDO USAR PARALLEL STREAMS

- ❑ Tarefas CPU-Bound: A operação em cada elemento deve ser computacionalmente intensiva.
- ❑ Grandes Volumes de Dados: O overhead do paralelismo só se paga em datasets grandes (Modelo NQ: $N * Q > 10.000$).
- ❑ Fontes de Dados Eficientemente Divisíveis: ArrayList, int, long são ideais. HashSet e TreeSet são razoáveis. LinkedList é péssimo.
- ❑ Operações Stateless e Associativas: As funções lambda não devem depender de estado mutável compartilhado. Operações de redução devem ser associativas.



ARMADILHA #1: O PERIGO DE I/O BLOQUEANTE

- ❑ O Problema: O `commonPool` tem um número limitado de threads (geralmente `num_cores - 1`).
- ❑ Saturação do Pool: Se uma tarefa no stream faz uma chamada de I/O bloqueante (rede, disco), ela monopoliza uma thread do pool.
- ❑ Consequência: Se muitas tarefas bloquearem, todo o `commonPool` pode ser esgotado, paralisando todas as tarefas Fork/Join e Parallel Streams na JVM.



ARMADILHA #2: LAMBDA COM ESTADO (STATEFUL LAMBDA)

- ❑ O Problema: Streams paralelos não garantem a ordem de execução.
- ❑ Condição de Corrida: Se a função lambda modifica um estado compartilhado (ex: adicionar a uma lista externa, incrementar um contador), ocorrerá uma condição de corrida.
- ❑ Consequência: Resultados incorretos, inconsistentes ou exceções em tempo de execução. As operações devem ser stateless.



CONCLUSÕES E MELHORES PRÁTICAS

- ❑ Padrão: Use streams sequenciais.
- ❑ Otimização: Considere parallel streams apenas para operações CPU-bound em grandes coleções eficientemente divisíveis.
- ❑ Meça, Não Adivinhe: Sempre faça um benchmark para confirmar o ganho de performance.
- ❑ Evite: I/O bloqueante, lambdas com estado e fontes de dados inadequadas (LinkedList).



ATÉ A PRÓXIMA!